ActInf MathStream 012.1 ~ ELBOing Stein: Variational Bayes with Stein Mix

MathStream | ELBOing Stein: Variational Bayes with Stein Mixture Inference, Ola Rønning | 58:17

Hello, welcome. It is April 10th, 2025. We're in Active Inference Math Stream 12.1 with Ola Running, going to be discussing Elbowing Stein, Variational Bayes with Stein Mixer Inference. So, thank you for joining. Looking forward to this presentation and discussion.

Great, thanks for having me. I'm Ola Running presenting the work I've done with Eric Nolestek, Christoph Lai, Frederick Smith and Thomas Hammerruck on Variational Bayes with Stein Mixtures.

So let's get into it. So, this is the plan. We'll talk about patient inference and particle-based methods and particular Stein-based methods which came about in 2016. I'll just go over an overview of what these are to get everybody up and then the key problem with them which is the underestimated variance as our models get high dimensional. Then I'll talk shortly about our approach which is Stein Mixture Inference. The new thing here is introducing a neighborhood to each other particles. We'll talk a little bit about that and then evidence that this mixture actually helps with the issue of dimensionality, of course, of dimensionality. And finally, Stein Mixture Inference is a black box inference engine in the probabilistic programming language of Pyro. So I'll plug that a little bit at the end.

Yeah, so just to set the stage here. So making predictions and sort of modeling systems is ubiquitous. So the group I'm in works with structural biology, but I'm looking to applications in robotics and potentially finance as well. And sort of at an abstract level, what we're looking for is a trustworthy systems. We want something that's accurate when it meets, when our data meets our expectation and uncertain when it's either surprising, ambiguous or missing. So illustrating it here by where we have a data generating process dotted line, and then we have a method which is trying to infer to infer the process. This is the red line. And so we only give it sufficient data in the green regions. And what we're looking for is something that captures the process in the green region and gets uncertain in this in between region. And so on the left hand side, you would have something that's accurate, but it's but it's overly confident because it the uncertainties is collapsed in the in between region. And the right hand side is what we're what we're what we want. So here, the blue region could be any potential curve. And it's like 95 or 90% HDI. So it's like the likelihood there would be in here. Good. Yeah. And so just to make it slightly more concrete. So in prediction, you might have something like a overconfident, overconfident mobile, mobile robot. So if you for example, have a robot navigating North a tree, it will have a sort of uncertainty about its orientation and position. But as you put it into new terrain, for example, by removing a tree, you would want this uncertainty to spread out so that you can invoke emergency protocols and then overconfident protocol would sorry, overconfident robot might lead to crashes because it's there's a disambiguity between its actual location and where it believes it's at. And this is a this is a we have a talking with some underwater robotics labs and they say that this is an issue in one out of

missions. And this can be quite expensive with robot costing between 20,000 euros and 10 million depending on the

size of the robot or deep sea robots. Another issue. So that was sort of when we're talking about prediction,

but when we're talking about modeling protein structures, three dimensional protein structures have regions which are

conserved. So they stay ordered so they don't fluctuate very much. But it's it's a it's a molecule interacting with the

solvent around it. So the whole thing will move slightly. And then you have some regions which are unstructured

and highly disordered, and they will be moving around. So if you want a description of the structure, it's not enough of

the three dimensional structure to protein, it's not enough to know just sort of image of the structure because there's might be some areas

which are not which will only be very or very unlikely to be in that particular position. So you want some sort of

representation which captures that you have flexibility in the molecule and uncertainty as a natural. We are

representing this. Okay, so and the way we are going to go about this is with Bayesian inference. So just a short reminder of Bayesian inference.

So we're given observations x. So this is the thing we get to see. And we have a model with latent parameters,

unseen parameters, which I'm going to call theta throughout. And then we can apply Bayes rule to get to take our

beliefs, prior beliefs about theta and incorporate the observations we're seeing to get to our posterior.

And we commonly use this posterior to either want to sample from it, which is the techniques I'm going to show you here,

or in some cases, we want to compute expectations with respect to the posterior. Now we have two challenges.

This first challenge is key, like it's important in terms of what we're doing here, which is our model of data is high dimensional.

And the other problem, which the thing you work around, you're doing this type of algorithmic is you don't want to evaluate the

norm length as in constant because it's interactable, analytically interactable at least.

So this methodology sort of first came out, came about in 2012. And the key idea is to take a simple measure and then find a push forward.

So a deterministic map such that you transform this very simple measure into the target measure.

And our target measure in Bayes' inference setting is our posterior.

And so this is one aspect of it. We're trying to find this deterministic map.

And secondly, we're going to move not the entire density because that would require us to compute

anomalizing constants. We're only going to be moving particles.

So we're going to move something that looks like samples from this simple distribution towards target distribution.

And so Moulti and Masuk did this using orthogonal polynomials.

So like they have this basis of orthogonal polynomials and then they found one constructed one function, which did the move the entire transport.

That was in 2012 and then in 2016 Steinberg and Green descent came out.

So the way they find that they determine the transport is they choose a distance between distribution.

So the simple distribution road and the target distribution posterior.

And then you apply the transport. And so they, Kjong and D-Link did two things.

The first thing was they deconstructed the transport into an iterative process.

And secondly, they sort of gave a simple form to each of these transports, which is just a perturbation to, to identity map.

And then after iterating this process, they would then make sure that each iteration you would get closer to, from the, from the, from the, from the, from this sort of simple prior metric towards the stereo metric.

And, and the key result here is that they get a close form for the sort of finding this vector field, which is making, taking, giving us the direction of our transport.

So just checking this apart, the name apart, we see the Steinberg version of gradient descent.

So it's a gradient descent algorithm.

We get a update rule, which looks like the classical gradient descent only.

Our, our dedus here are random variables and not the parameters that we would be used to.

And the way it works is we draw a set of particles from, from our prior, this set of particles, we put an empirical measure on.

And then we compute, we find a minimizing distance in terms of KL towards, for the next step.

For the next step and we apply that transport so that, that, that, that vector field to the particles rather than the entire density.

And this eventually converges to our target, target density.

And then there's the Stein part.

So this is going to get us, get us the direction in which to move particles.

So what they find is they find a direction, which is the maximum decreasing direction in the, in terms of KL.

Where the vector field is of contained within a large enough class of functions, which are called big by.

Okay. And so this was known before, but there's a relation between the KL divergence.

So this is known after this is

known after this is known after this is known after this is known after
questions about this before we've oh this is an old version sorry the score function allows us to
this should be a theta down here and then we don't get to evaluate we don't have to evaluate the
um the gradient respect to the evidence sorry derivative with respect to the evidence that
will just be zero so we can evaluate the score by just having a joint uh over theta and x over a
latent and our observations and the second insight uh that one was sort of well we can have a form for
the for the for the vector field but we can actually also compute it if we choose the space correctly
uh oh that should be second uh and and what would the space of functions that we're choosing is
reproducing kernel of hilbert space uh and where we are what we get is with a kernel is the partial
evaluation is a point in the hilbert space so if we have a way of if we have have kernels available
to us with simple functional forms then we also have a way of computing computing the optimal direction
um and and kernel hilbert spaces or sorry uh reproducing kernel hilbert spaces have sort of been
studied both in in in gaussian processes and uh and i think support vector machines before this so we
have quite a few available um the one we're going to be focusing on here is the squared exponential kernel
uh function form is given here uh there's quite a few different ones there's also so these are
the scalar kernels and what you do is you can take the identity uh matrix to make it to into a uh uh
a vector field over uh over rd there's also matrix variants and then we have operators which can
allow us come to combine kernels and there's a lot of equations here and i sort of this is a simpler view
of the whole thing so uh if we we have a kernel and it's it's partial evaluation gives us a kernel in
the function hillbots uh in the reproduction kernel and helbert space so for example a constant for a
kernel um that just our constant will decide exactly which value we take um in a linear kernel you would
have something like um the kernel sorry the the partial evaluation which everyone would choose will
choose the the slope of our function and the key one we'll be working with is the square financial also
known as the caution and there the partial evaluations uh give us the mean whereas we have a a hyperparameter
we have to set ourselves which is the bandwidth and that is going to choose the the the width of our
of our caution can you just clarify what the red and the blue are are for these different kernels
yes i'm taking a kernel right a kernel is a function in this case it's just r goes to r goes to r so x here
is a free variable and and and and our number is a fixed one of the parameters and so uh the blue dotted
line corresponds to fixing one of the parameters to two the gray one is fixing it to one oh sorry to zero
and the red is fixing it to two two minus two is there a reason for two and negative two or is this
just showing like the two directionalities you can take from zero or is there something about the two
no no there's nothing particular about choosing mine it's choosing both sides of uh when we just
have when we just have a line right uh it was just to show illustrate sort of what happens with the only
interesting boundary here which is over zero awesome so they're like the two directions you can go
updating the kernel yes if we're in one dimensional space
and then they all have hyper parameters like a length scale or uh which i've shown down here but
they also just chosen arbitrarily just to illustrate uh this functionality

all right so uh recall that i showed you this uh this functional form of the update just writing it out uh with expectations so right now i'm just applying it to two or two particle system we get us it will get something that uh that behaves like we see this vector field actually behaves like forces acting on physical particles so the first term is effective uh force it's moving towards the nearest mode in our posterior and it's moving each of the particles like we're taking the it's a mean field method right so we're taking the average uh effect uh and moving in the particle according to that so we have the attractive force in blue and then we have a repulsive force which acts pairwise between particles and that is keeping the particles apart so they will have uh so they will not collapse onto each other if we only had the first uh the first term here the attractive force we would have no benefit of having more than one particle uh because everything would just collapse through the new mode in our final system so we initialize like these positions randomly we run this system forward and what we get is something that looks akin to samples from the posterior they are not exactly samples because they which is also what we call them particles they have they have correlation structures which are sort of different than you would have for mcmc but they do such certain retain some sort of correlation structures i don't think it's well characterized though at the moment

so uh so that's sort of gives an idea about a particle approach and so uh particle approaches like in one dimension these they're short like they are as good as good as mcmc but as we go up in dimensionality we start seeing issues so yeah i'm doing the same system i was doing before i'm using a a bayesian neural network which has in the low dimensions 41 dimensions and a high 10401 dimension and we capture some uncertainty in this in-between region when we're using in the load uh in the low dimensional system but as we move up in dimensions this completely collapses

so now the order based network sort of how many parameters also decides um um how expressive our model is and so uh so as you can see with the ideal you get less expressive like you there's fewer curves you're actually able to fit uh with a lower dimensional bnn which is why you see a difference in terms of uncertainty in the ideal is going from low to moderate so uh interesting question then is well when do we sort of stop trusting stein version of great descent and uh jimmy bha in 2001 came up with an interesting experiment where you have um jika gaussian we let it stemmed multivariate gaussian let its dimensionality grow slowly and then you're looking at sort of what is the average uh uh empirical variance you get per dimension uh of your estimate and what you see with the inversion of grain descent is already at five dimensions you've sort of halved this uh marginal variance uh and so you should really not trust uh uncertainty estimations if you have a problem which is five or higher dimensional and there's a lot of properties how does that relate to the factorization like the difference between doing five parameters in a joint distribution or factorizing some of the variables apart

i'm not sure i completely follow like when you're talking about the dimensionality increasing this is doing a joint distribution of all of those okay so in their possible mutual influence

yeah so here we're actually so our um our multivariate gaussian is assumed completely independent which is why we can compute this very simply easily so it was interesting question whether or how it actually works if you start having covariances as well but this experiment is really the simplest version

you could imagine where you just have a okay and and if there were covariances do you think that would make the situation worse or more attractive i would assume i would assume it would make it worse okay this is sort of the simplest version of the system i could imagine uh so this would make it worse so this is sort of a lower bound on what we're uh what we should expect

um yeah yes good so onto our approach sign mixture inference so what we uh propose is to separate the particle space and the latent parameter space so that you get a hierarchical model and you let the uh the the particles move in this uh in the site uh space and then you let each particle parameterize its own rational distribution which then exists in the same space as our posterior and so what you get is you get a mixture approximation of the same uh rational distribution uh to our posterior and uh here m is the number of particles so it would be two so if you're using 10 particles then you're using uh then you would have a 10 component mixture approximation

and just to motivate it uh we can rethink so we can rethink the the the Steinberg should be in the send particles as sort of the take if you could imagine taking a Gaussian and then letting the variance go to zero you would get something that's just the direct delta function uh distribution but it would just be like a a probabilistic atom and so you can think of in this sense you could think of uh sort of a Stein particle as just the view location of this sort of degenerate Gaussian and so going in the opposite direction you could say well if we then uh allow the variance uh of uh of them of the normal to also be represented by the particle then each particle now becomes two-dimensional um and and and and summarizes this Gaussian so so we this our particles now both have a location but they also have a complement for the variance um and then with multiple uh like having many of these particles you suddenly get a mixture approximation and in this case a Gaussian mixture approximation and so our goal now is to be able to move these particles uh in the in in in such a way that we're sort of minimizing a distance to the uh to the posterior to get a base uh version of base that's it and to do this we're going to look back to a sort of mean field version of inference and just to recall here you have a you have a simple parametric distribution so we're sort of not we're not necessarily trying to hit uh hit uh get the shape right of our posterior however we are gonna work with something that's uh that is uh tractable and so here you were just a Gaussian summarized by this this midline it's mean and we sort of then move the mean such that it's covering as well as it can uh the posterior distribution and we'll do this with a proxy distance um so it will not be the kale we're minimizing anymore it will be a evidence load and so just to recall that as well it's it's a our evidence lower bound is just a difference between our evidence of yeah log of data and the kale between our approximate uh distribution cube and our true posterior and it's it and it's this expression and the reason we work with this is this this expression here which is very easy to compute especially if we have a very simple rational distribution queue we might even be able to compute the the differential entropy analytically

good so the idea of so as i mentioned we're gonna take uh our particles and give them a neighborhood so in terms of of uh of the equation i was showing you before we could think of making these uh the parameters uh of our rational distribution we can

make those into random variables and if we do that then uh we would get uh and we had multiple of them and we
have uh we would have an empirical measure on uh on the parameters of our of our rational distributions and it would be the same as replacing the score function in the stein version of gradient descent uh with with some sort of uh of loss function here which is going to be similar to an elbow now we'll have to do a bit of massaging to make sure that this is still a valid uh math methodology but sort of in its sort of simplest form this is all we're doing we're we're moving taking our transport we are moving on these particles which are now a hierarchy higher than it was before and we are moving them using uh using this uh this elbow formulation instead uh of in terms of the of the score function and the mic uh the evidence law about that we're using is takes this particular form you could use simpler versions of this but it actually it behaves doesn't behave well so we need to take in particular we need to take this uh the differential interview in terms of uh of our rational approximation not just each of the terms
of the terms um in order to show that this is correct we do have a constraint on this which is our guide distribution operational uh uh distribution q that needs to be the same uh for every single particle in other words our our function of functional of row uh needs to be symmetric so it can't depend on the the ordering in which you uh in which you uh parameterize it
with this change uh we can use uh an extension of
the integration gradient descent called non-linear stein version gradient descent and what this is again dealing on jointly what they did was they replaced so it's time version of gradient descent we're minimizing the kale divergence and what this extension does it is allows us to minimize um functionals of our initial distribution row uh with our uh differential entropy as our as a regularizer and what that is doing is sort of ensuring uniformity so ensuring that our our density row gets spread out there's some prerequisites for this first of all as i already mentioned uh our functional when uh parameterized by uh uh empirical measure needs to be symmetric secondly we need to have a differential evaluation map oh this is again the old version i'm sorry this should be this is all the way up to m there's nothing special about m here um but we need to be able to to evaluate uh the functional given our particles and using an empirical measure that's very big because it's just evaluating at each other each of the points and finally here we have this regularizing term which is ensuring that our particles are spread out
so our uh as we show okay but this is uh we have an instance of non-linear steinberg's gradient descent first of all it fulfills that the particle approximation is symmetric because we chose the same q for each of our each of our particles so the same uh guide we have a differentiable map but it's trivial to evaluate on the empirical measure and if we choose it needs to be differentiable so we also choose to our guide to be differentiable and finally we were when i showed you we were replacing sort of the inner part of the of the vector field by uh evidence lower bound but recall we are now this is
the objective we're actually optimizing is the dysfunctional plus some entropy to monroe which is up here and so we need to show that that is also an evidence lower bound in order to have a valid variational uh approach and if you choose so they have this alpha parameter if you choose that to one

and you look at the population limit so infinitely many particles and i'm moving this uh then we actually we can actually show that this is the case so it's a valid variational base method
and so if we revisit the experiment i was showing in the beginning um with uh with sign version gradient descent we had collapse and now with our sign mixtures we see that average going up in dimensionality our particles uh uh are in in theta space so in the sort of posterior space
uh our product we now have uncertainty estimation in this uh in the moderate dimensional case as well however it's not uh it's not ideal yet uh we are here we're using five particles and so you can get better approximations using more but the cost of the sort of the cost of each step is uh quadratic in the number of particles so you want to use as few particles as possible um but but there's so there's there's still room to be uh for improvement uh but but we are work it works better than than uh than stein version gradient descent for this uh uncertainty estimation
so as i said we we sort of there's still improvements to be made there's some key design dimensions that we can address first of all dynamic stability which is we sort of
stein version of gradient descent has been understood as a gradient flow in
in in the space of distributions however if you're just doing that naively with mixture models then you might your gradient flow might move somewhere which is no longer a mixture model so we in order to sort of analyze and understand uh uh uh stein mixture inference you need some some new theory there uh but with this if you get this gradient uh low uh if you nail that and you start being able to do things like you can you can choose hyper parameters by choosing different formulations of the gradient flow uh and and and for example look at uh the vessel thing flow versus uh this is getting to digger sorry uh um what you can do you can start uh determining something like kernels based on having uh this type of formulation it also allows us to tell things about how fast it converges which is currently not now uh the result on steinbridge screen descent at ic law this year which shows that it's in ksd it's actually as good as uh as uh mcmc so that's very good in terms of ksd and there's kernel designs like actually choosing the the kernels so one way to do this is knowing the actual uh dynamics are uh but also then understanding based on which kernel you choose how does this uh affect dynamic stability and uh duncan has an interesting paper on that uh for steinbridge gradient descent which i believe can also lift two time mixtures finally there's accelerating uh acceleration options like newton methods free conditioning of gradient flows to completely avoid uh computing uh kernels
yeah so that's sort of a rough outline of time mixture inference uh it's available in numpyro the deep probabilistic programming language uh it separates model from inference it has a dsl for probabilistic programs embedded in python so you can have arbitrary python in between random sites it uses non-standard interpretation and most importantly like for this talk i guess is that stein mixture is a black box uh inference engine in the system um of course we need to be able to take derivatives hardware acceleration linear algebra and all that is handed off to jacks so it's just built the layer on top of jacks
yeah there's been a lot of people involved in this so first of all the pyros devs that helped me build the the um the inference engine martin jankowiak and uh you phone and akhmet al-sabahi who did the first

version of it uh which then evolved into our um into our uh the current version of stein bi which is
the name of the inference and engine and then my co-authors so a list nick who first formulated this
problem uh when he was a phd with patrick smith and then thomas hammeruk was my pi um his work with me
for the last couple of years and christoph leigh who's an expert in stein's method at university of
luxembourg the key references sum up so if the introduction into this approach to patient inference
uh is the first one then there's the two stein papers from 2016 and 19 and then our uh my paper here
which is going to be at ic law this year yeah it's the inference group and uh at ku yeah thanks
awesome thank you all right i'll ask a few questions and then if anyone watching along
wants to ask a question i'd be happy to so to kind of pull back and re-summarize
what makes kl divergence optimization for particle transport plausible where distributional transport
is not plausible uh so the the key issue here is computing the normalizing constant
you need you need to be able to re-normalize uh re-normalize the density so
like if you choose a very nice space of functions such as so they were using this orthogonal uh
polynomial basis then we we can do it but it restricts sort of how expressive you are able to do
be in terms of what you're transporting a particle methods you so if you it scales proportional
with the number of particles it's cheap to compute great so you you mentioned that the compute is
quadratic in the particle number so what are the computational complexity considerations for this
method and then how does that even kind of qualitatively relate to the mcmc approach like
if that's giving a better posterior estimate of the real paths what are the conditions where mcmc
is just straightforwardly a better method to use versus exploring how many particles etc for this
method so uh so first in terms of mcmc i was like in a toy example you would probably just use mcmc but
it becomes truly intractable if you sort of if it's the issue if especially using hmc or something like
this is that it's because it you need to work on all data points right so because you need to compute
the gray like when you're doing it's in plexic integration you need to compute at every step you
need to compute the gradient with respect to every data point and that's why mcmc just doesn't that
doesn't scale to the type of data the data sets we're sort of interested in um in terms of how is the
complexity with um compared between stymversion gradient descent and smi the key uh sort of the expensive
part is computing the kernel you need to compute the pairwise components this is why it's n squared and
we have the same kernel structure uh for svd and smi so the difference here is that we can what you can do
with five particles with stein mixtures you might not at all be able to do on conventional hardware
uh with the stein version uh version of great descent if your problem is high dimensional enough
okay so from the live chat so first quick question from gary wrote what language are you using
so you mentioned the software package just could you re-describe or even show the software package or
what programming language is it in and what does that domain specific language look like okay uh yeah
sure it's so the first of all the language is in python uh and it's um maybe should i share something else
yeah sure

one second

something about that

something about that let me show you

all right

all right

now let's this ah man here we go

okay

okay yep

so can you see it

yep it's a little small if you can zoom in but we see it

yeah okay we'll have to do like this uh we can see it all right good so this is uh this is uh

uh a va so a variational uh auto encoder right so what you're doing is you're taking you have

uh you have a you have a guide which takes a picture or something down to some uh some to some low dimensional model so we call this the the encoder and then we have a model which takes our latent low dimensional and back okay and so what i'm showing you here is uh is the the the decoder so what takes our latent representation and moving us back into the original uh original space so you can think of this as compression maybe

um okay and so

see

this is what the random sites look like so you have you have a random variable here uh we're gonna be we're gonna be using um

a non-standard interpretation so like have some have a set of uh operators which operate on the sample sites or handlers which operate on the sample sites and make them behave

differently than otherwise just straight executing like them so here i have a random sample site

it's over angles it's distributed according to a sign by a variable on my system distribution so there's just a normal think of it as a normal on a torus it has some parameters and then uh i get to i have some observations so this is like conditioning the same as conditioning on these angles that i observed now

these uh parameters for my sign by my sins uh when my sis i have some locations and concentration these come

in this case from applying uh a neural network or patient neural network it's a um and and that's just coded up in flags so that's just straight up standard uh patient net uh so uh neural network and then we sort of we can we can uh make it into our patient network by just introducing a prior over it's uh over it's over the width so this is what's happening here making a patient neural network we're saying we have a uh uh the current neural network uh it's a random site called decoder so we can apply our handlers uh to this site uh and then uh this this just make sure that uh it's the right the parameter like the right parameters of being updated when we're doing our equation uh updates okay so and that then uh given some some some latent remember they said uh they gave us my my uh some representation which i can then

map into the parameters my sign barrier from my six so just give you an idea about this is what the

like what modeling looks like we introduce uh random sites and we can do sort of arbitrary things in python in between our random sites to manipulate whatever uh samples without that's the sort of the one side of it and then when you then want to do inference well you're so we're setting up our inference method so right now i'm just testing thing out so it's just mean field operational inference i have the model which is what i just showed you i have a corresponding guide so this is my queue i was talking about this is the thing encoding my uh my space into this latent representation i'm going to decide on the typo optimizer using atom and then i'm saying i'm going to use uh just a standard uh evidence lower bound here um index or in numpyro we need to be explicit about our the state of our random keys that's what's going on here and then i'm running my inference so now i'm conditioning uh my uh the parameters the parameters of the parameters of the guide on the observations that i have available which are stored in phi and psi and from there i just dump the object and i have a separate thing which does my diagnostics so i can get my predictive distribution out i can see what the stupac shandran plot look like uh if i want to look at uh histograms of parameters i can get those out from my predictive but that's sort of the the high scale view of a typical implementation or project

awesome okay next question in the chat bert burkers wrote in the slide showing attraction and repulsion there are two particles with vectors pointing in many directions how should one interpret these directions and their angles should i yes this is perfect good okay so the uh the blue uh the blue arrow will always point towards the closest uh mean so it's if we're looking at a gaussian distribution here and it's just in a mobile right so the particles are just the closest mean will always be the center of the country um so that's what the blue arrows uh uh are showing you and then the the if you aligned up uh the red arrows there are the forces acting on the particles by each other and so they will all in a two system uh particle system they will always be plus and minus at least if you choose uh uh gaussian kernel of the same expression so they will be of the same magnitude you have three particles it becomes more complex and then the black and i should have mentioned this that that's the net effect of the two okay so it's looking at when you add up these two uh these two components then you get the net effect of the black the black arrow so that's when you're then taking a step you're moving in the direction of the black arrow and then like depending on how you choose your epsilon you might not take the full length of this of the course field but some epsilon size of it and just to kind of clarify what tunes the relative strength of the attractive and the repulsive force like at the end of all of this procedure how do you ensure that you haven't over or underestimated the variance envelope

so this particle like this system of equations will when we are at convergence this will there will be no force acting on any of them right and so this this this equation will be zero

got it so the particles

engage in these attractive repulsive

physics-like interactions amongst observations delta-like kind of hidden gaussian observations that can be variance reinflated and that particle transported particle distribution it's almost like recapitulating what the mc would give you from samples because you do want a sample from a posterior distribution with a finite number of spikes and instead of actually just doing that gradient calculation at the data point level

as done in mc here you're learning these transport functions with certain conditions that lets you move the distribution
project down to particles and reinflate the particles back in your contribution to this mixture model
yeah so we're we're moving the particles so that they get towards the posterior so like the data is
of all our observations are hidden in in germ in steinberg in descent at least in in in our score right
okay i guess just taking one step back and then if anybody else wants to write any questions
how did you come to this problem for your postdoc like what led you through studies and everything to
want to target this uh yeah so yeah so actually it was uh it was uh
it was uh akward who who so uh my original pitch my phd was on uh i'm sort of trying to find uh
to determine uh three-dimensional structure of proteins given um given the sequence so the pro uh protein
3d structure problem and and when i when i started uh and we had uh alpha full come out sort of two months after
um which made the which and what that sort of that was declared as a solution to the problem which made it
uh a bit uh a bit of an interesting uh situation to be in but uh but akmat had started working on this
um because he thought it mapped well onto so so one thing we want right is we want to be able to
uh with our with the probabilistic probabilistic program is we want to automate uh automate inference and we
sort of we know that there like this can't be done entirely but we want something that's very expressive
and cheap to compute with and and it's time version gradient descent had had these two active properties
it's very cheap and easy to understand uh uh and it's cheap to compute with so we sort of we built a
stinger should be in the scent uh inference engine and then we were looking at well we have this issue
and we wanted to apply it to proteins uh eventually and so what i was showing you before is well now i'm
getting to that phase because it turned out to be a lot harder than we first expected to actually
getting the inference engine running was not as much work and then finding out is this thing correct
uh and that's what we spent the next while on a couple of years
there's always a funny postdoc story um what does it look like to go from the open source code that you
have to adapting it to a problem like what's the kind of recipe that somebody would take
from from um that's one question and then a second question however you want to do this is where do you
see this method in terms of the landscape of like generative ai pure neural network based approaches etc
i'll take the first one uh first so what we would do there is we would go uh we would
do it
and okay so we would import our method instead of instead of uh instead of uh instead of svi and then we
would replace it here
and now of course we need to choose a kernel so just do the rbf kernel
and that that's it okay just just to confirm that move because it's largely built upon the jacks pyro
approach it's not so different than just flipping an import for a model that's already written within that
way yeah okay so how about how how does somebody even if this is sort of a basic question for pyro users
let's say somebody's looking at you know the temperature and the humidity in a room
and making a kind of active inference model where they're going to run a heater or a cooler

or some other kind of well how do we go from just the observables and the unobservables of our target system of interest into representing it in a way that allows it to be inferred using this probabilistic approach

yeah so so what you're describing sounds like the sort of the patient network or factor graph uh right and that's the model i was showing you is how i write that off so i like whenever i have a random variable in this case this is not the greatest of examples to look at but uh but i guess i have a set right so i have a random variable here which is my latent and then i'm saying that that there's an arrow between set and and uh and my angles right and that's the same as saying like i have two uh i have a graph structure actually i can't show you stuff telling you um let's see so this is the sort of view that i think you were thinking of like we have random variables uh uh sort of here we have set and we have another one which is observed this angle but that's just that's just described as a python function which is model which is using this special uh specialized um uh like uh effectful uh methods one of them being sample one another one being what an example corresponds to to a circle here uh plate corresponds to what you would know in the plate in a patient graph and then we have deterministic so these are these uh dotted line circles awesome and is this generated or specified in another file in this repo how the bv model yeah yeah that's so that's just so the thing i was going at first this model is just that it's specifying this thing and it's just written as a function and that function i then act on with uh with my particular inference engine awesome okay so then kind of again pulling back like where do you see these methods in the toolkit of what people could do with with pure neural network approaches or transformer based approaches today so uh so i mean so the patient neural network is sort of lifting this up right and like pretending that we can actually get access to a to the posterior of its parameters instead of point estimates and and there are i think i i like that your vision neural networks because exactly because things of at least seems like they could give us an idea about risk or uncertainty there's a couple of problems with it right like it's we have like moving you have you have two networks which look different in terms of weight but represent the same function so like whether we are actually whether patient networks are actually like yeah yeah whether we get when we're moving in weight space where we're actually getting different sort of samples even though we're changing uh changing our weights is not clear and so so so it's not really you can't really determine whether you get different uh different uh if you're running something like fcmc you wouldn't actually know that your particular samples are different networks necessarily so like estimating how well you've done or even determining convergence becomes very hard but it does handle sort of misspecification quite well right like if if you have something that's exceedingly flexible you probably have the right moment so that's why i think patient networks are attractive but we need to address this non-identify ability eventually and i might be able to do that with kernels but we don't know yet we have some ideas cool well is there anything else you want to show or add or like how can people other than reading the paper and checking out the repo learn more yeah i mean there's a this is where you get started so pirate are there take a look uh our version is the non-pyro version but pyro's uh also very uh large and uh very fully fledged uh deep probabilistic program language which i

hire and encourage you to take a look at cool anything else you want to add
okay awesome thank you again ola for joining super interesting i i hope we can continue to learn and that people in the institute community explore like pyro cure methods rx and fur we can start to see where these different methods complement each other top differences all that cool thank you bye thank you

Thank you.